데이터사이언스

Lecture 10 데이터 전처리

왜 전처리가 중요한가?

- Garbage In, Garbage Out — ML 성능은 데이터 품질에서 시작된다

80%
데이터 과학자가
전처리에 쓰는 시간

40%p
전처리만으로
향상 가능한 모델 정확도

3대
대표 문제
결측·이상·인코딩

머신러닝 파이프라인에서 전처리의 위치



결측치(Missing Value)란?

- 데이터셋에서 특정 관측치의 값이 기록되지 않은 상태
- pandas에서는 NaN(Not a Number) 또는 None으로 표현됨
- 설문 미응답 / 센서 오류 / 데이터 병합 실패 / 수동 입력 누락 / 시스템 오류 등 다양한 이유로 발생
- 대부분의 ML 알고리즘은 결측치를 처리하지 못함
- 무작정 삭제하면 데이터 손실, 잘못 대체하면 편향 발생으로 이어짐

결측치 탐색: 현황 파악이 먼저

- MCAR, MAR, MNAR

```
# ① 기본 확인
df.isnull().sum()                  # 열별 결측치 수
df.isnull().sum() / len(df) * 100  # 결측치 비율(%)
df.info()                          # Non-Null Count 한눈에

# ② 결측치가 있는 열만 보기
missing = df.isnull().sum()
missing[missing > 0].sort_values(ascending=False)

# ③ 결측치 있는 행만 보기
df[df.isnull().any(axis=1)]

# ④ 열별 결측치 비율 → 처리 전략 결정
missing_rate = df.isnull().sum() / len(df) * 100
print(missing_rate[missing_rate > 0])
# 비율 < 5%   → 대체 또는 삭제 모두 가능
# 비율 5~30%  → 통계적 대체 권장
# 비율 > 30%  → 해당 열 삭제 고려
```

결측치 처리: fillna()

- 상황별 대체 전략

```
# ① 수치형 – 평균 / 중앙값 / 최빈값
df["나이"].fillna(df["나이"].mean())        # 평균 (이상치 없을 때)
df["나이"].fillna(df["나이"].median())      # 중앙값 (이상치 있을 때 권장)
df["등급"].fillna(df["등급"].mode()[0])    # 최빈값 (범주형)

# ② 시계열 / 순서 데이터 – 보간(interpolation)
df["기온"].fillna(method="ffill")           # 앞 값으로 채우기
df["기온"].fillna(method="bfill")           # 뒤 값으로 채우기
df["기온"].interpolate(method="linear")     # 선형 보간 (앞뒤 평균)
df["기온"].interpolate(method="time")       # 시간 기반 보간

# ③ 그룹별 대체 (더 정교한 방법)
df["나이"] = df.groupby("성별")["나이"].transform(
    lambda x: x.fillna(x.median()))         # 성별 그룹별 중앙값으로 채우기

# ④ dropna – 삭제 전략
df.dropna()                                 # 결측 있는 행 전체 삭제
df.dropna(subset=["나이","소득"])           # 특정 열 기준 삭제
df.dropna(thresh=int(len(df.columns)*0.7)) # 70% 이상 있는 행만 유지
```

결측치(Missing Value) 예제

- "week10_example.csv"는 병원 환자 20명의 데이터이다. 변수 설명은 다음과 같다.

변수	설명
이름	환자 이름
나이	환자 나이 (세)
몸무게	환자 몸무게 (kg)
혈당	공복 혈당 (mg/dL)
진료비	진료비 (원)
성별	환자 성별 (남/여)
등급	환자 등급 (BRONZE/SILVER/GOLD/VIP)
입원일	입원 날짜 (YYYY-MM-DD)

- 1) 전체 결측치 개수는?
- 2) 나이 열을 평균으로 대체한 뒤 남은 결측치 수는?

이상치(Outlier)란?

- 이상치란 데이터의 전반적인 패턴에서 크게 벗어난 값

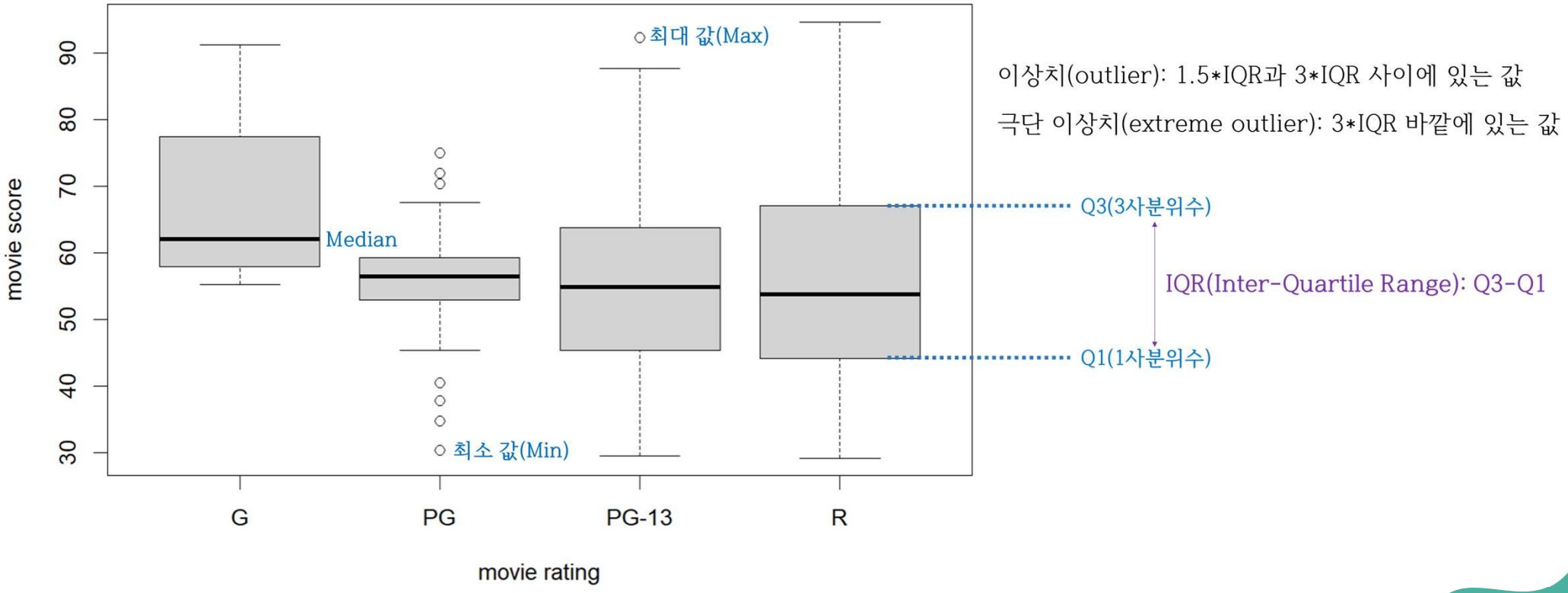
	정상 데이터	이상치 예시
나이	20~60	999
키	150~190cm	3cm
시험 점수	0~100	-50
매출	100만~500만	10억

- 발생 원인
  - 입력 오류: 사람이 잘못 입력 (999, -1 등 placeholder 값)
  - 측정 오류: 센서 오작동, 장비 이상
  - 데이터 병합 오류: 단위가 다른 데이터가 합쳐진 경우 (원 vs 달러)
  - 진짜 극단값: 실제로 존재하는 특이한 케이스 (초고소득자 등)

```
normal = [20, 25, 30, 35, 40]
with_outlier = [20, 25, 30, 35, 40, 999]
```

이상치(Outlier): IQR 방법

- 사분위수(Quartile): 데이터를 크기 순서대로 정렬했을 때 4등분하는 기준점
- Q1 (1사분위수): 하위 25% 지점, Q2 (2사분위수): 하위 50% 지점 = 중앙값(median)
- Q3 (3사분위수)하위 75% 지점



이상치(Outlier): IQR 방법

```
Q1 = df["나이"].quantile(0.25)    # 1사분위수
Q3 = df["나이"].quantile(0.75)    # 3사분위수
IQR = Q3 - Q1                     # 사분위 범위

lower = Q1 - 1.5 * IQR            # 하한선
upper = Q3 + 1.5 * IQR            # 상한선

# 이상치 확인
outliers = df[(df["나이"] < lower) | (df["나이"] > upper)]
print(f"이상치 수: {len(outliers)}개")

# 이상치 제거
df_clean = df[(df["나이"] >= lower) & (df["나이"] <= upper)]
```

이상치(Outlier): Z-score 방법

- Z-Score: 어떤 값이 평균으로부터 얼마나 떨어져 있는지를 표준편차 단위로 나타낸 수치
- $Z = \frac{X-\mu}{\sigma}$

```
from scipy import stats
#z-score 계산
z_scores = stats.zscore(df["나이"].dropna())

#이상치 필터링
outliers_z = df[abs(z_scores) > 3]     # |Z| > 3 이면 이상치
print(f"Z-Score 이상치 수: {len(outliers_z)}개")

# 이상치 대체 (제거 대신 경계값으로 클리핑)
df["나이"] = df["나이"].clip(lower=lower, upper=upper)
# → 하한선 미만은 lower로, 상한선 초과는 upper로 교체
```

이상치(Outlier): IQR vs Z-Score

	IQR	Z-Score
기준	Q1, Q3 기반	평균, 표준편차 기반
이상치 기준	하한선·상한선 밖	절댓값 3 초과
이상치 영향	거의 안 받음	영향 받음
적합한 경우	이상치 많은 데이터	정규분포에 가까운 데이터

이상치(Outlier) 예제

- “week10_example.csv”를 사용하세요.
- 1) 나이 열의 IQR 값은?
- 2) IQR 방법으로 나이 이상치 탐지 시 해당 환자는?
- 3) 혈당 열에서 IQR 이상치 탐지 시 해당 환자는?

데이터 타입 변환

```
# 타입 확인
df.dtypes
df.info()                                          # Non-Null + dtype 한눈에

# 기본 변환 astype()
df["나이"]   = df["나이"].astype(int)
df["가격"]   = df["가격"].astype(float)
df["성별"]   = df["성별"].astype("category")   # 메모리 절약!
df["코드"]   = df["코드"].astype(str)

# 문자열에 숫자+쉼표 혼재 → to_numeric
df["가격"] = df["가격"].astype(str).str.replace(",","").str.strip()
df["가격"] = pd.to_numeric(df["가격"], errors="coerce")  # 변환 불가 → NaN
```

데이터 타입 변환

```
# 날짜 변환 to_datetime
df["날짜"] = pd.to_datetime(df["날짜"])
df["날짜"] = pd.to_datetime(df["날짜"], format="%Y/%m/%d")  # 형식 지정
df["연도"]  = df["날짜"].dt.year
df["월"]    = df["날짜"].dt.month
df["요일"]  = df["날짜"].dt.dayofweek   # 0=월요일 ... 6=일요일

# category 타입 활용
df["등급"] = df["등급"].astype("category")
df["등급"] = df["등급"].cat.set_categories(["BRONZE","SILVER","GOLD","VIP"],
ordered=True)
print(df["등급"].cat.codes)              # 내부 숫자 코드 확인
```

데이터 타입 변환 예제

- “week10_example.csv”를 사용하세요.
- 1) 진료비 쉼표 제거 후 int 변환, 전체 합계는 어떻게 되는가?
- 2) 입원일 datetime 변환 후 3월 입원 환자 수는?

인코딩: Label Encoding

- 범주를 숫자(0,1,2...)로 치환
  - 순서 있는 범주에 적합 (등급, 학년 등)
  - 트리 기반 모델에 주로 사용

```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
df["등급_enc"] = le.fit_transform(df["등급"])
print(dict(zip(le.classes_, le.transform(le.classes_))))
# {'BRONZE':0, 'GOLD':1, 'SILVER':2, 'VIP':3}
```

인코딩: One-Hot Encoding

- 범주마다 0/1 열 생성
  - 순서 없는 범주에 적합 (성별, 지역 등)
  - 선형/신경망 모델에 주로 사용

```
df_ohe = pd.get_dummies(df, columns=["성별","지역"])
# → 성별_F, 성별_M, 지역_서울, 지역_부산 ... 열 자동 생성


# drop_first=True → 다중공선성 방지 (dummy variable trap)
df_ohe = pd.get_dummies(df, columns=["성별"], drop_first=True)
```

인코딩: Ordinal Encoding

- 순서가 있는 범주형 데이터를 순서를 지정해서 숫자로 변환하는 방법
  - 순서를 직접 지정하여 숫자로 변환
  - 순서 있는 범주에 적합 (등급, 학년, 만족도 등)
  - 트리 기반 모델 및 선형 모델 모두 사용 가능
  - Label Encoding과 달리 의미 있는 순서 보존

```
from sklearn.preprocessing import OrdinalEncoder
oe = OrdinalEncoder(categories=[["BRONZE","SILVER","GOLD","VIP"]])
df["등급_ord"] = oe.fit_transform(df[["등급"]])
# BRONZE=0, SILVER=1, GOLD=2, VIP=3 (순서 유지!)
```

인코딩 예제

- “week10_example.csv”를 사용하세요.
- 1) OrdinalEncoder 적용 시 SILVER의 코드값은?
- 2) 성별 열(남/여)에 pd.get_dummies(drop_first=True)를 적용했을 때 생성되는 열 이름은?

스케일링

- 수치형 데이터의 범위를 일정한 기준으로 통일하는 작업
- 머신러닝 알고리즘은 숫자의 크기를 기준으로 학습하는데 열마다 단위와 범위가 다르면 숫자가 큰 열이 모델에 과도한 영향을 미침.
- 나이, 연봉, 키?
- 스케일링이 특히 중요한 모델:
  - 선형회귀, 로지스틱 회귀
  - SVM (Support Vector Machine)
  - KNN (K-Nearest Neighbors)
  - 신경망 (딥러닝)
  - PCA (주성분 분석)

스케일링: MinMaxScaler

- 모든 값을 0과 1 사이로 변환(가장 작은 값이 0, 가장 큰 값이 1)
- (x - 최솟값) / (최댓값 - 최솟값)
- 사용 예시:
  - 신경망, 딥러닝 입력값
  - 이미지 픽셀 (0~255 → 0~1)
  - 값의 범위를 명확하게 정해야 할 때
- 주의: 이상치에 민감

```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
X_train = scaler.fit_transform(X_train)  # fit + transform
X_test  = scaler.transform(X_test)       # transform만!
```

스케일링: StandardScaler

- 데이터를 평균이 0, 표준편차가 1이 되도록 변환
- 값이 평균으로부터 얼마나 떨어져 있는지를 표준편차 단위로 나타냄
- $\frac{X-\mu}{\sigma}$
- 사용 예시:
  - 선형회귀, 로지스틱 회귀
  - SVM (거리 기반 알고리즘)
  - PCA (분산 기반 알고리즘)
  - 대부분의 ML 알고리즘 — 가장 많이 쓰이는 스케일러

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)  # fit + transform
X_test  = scaler.transform(X_test)       # transform만!
```

스케일링: RobustScaler

- 평균과 표준편차 대신 중앙값과 IQR을 사용하여 변환
- 값이 평균으로부터 얼마나 떨어져 있는지를 표준편차 단위로 나타냄
- $\frac{X-Median}{IQR}$
- 사용 예시:
  - 이상치가 많은 데이터
  - 정제가 덜 된 실제 현업 데이터
  - 이상치를 제거하기 어려운 경우

```
from sklearn.preprocessing import RobustScaler

scaler = RobustScaler()
X_train = scaler.fit_transform(X_train)  # fit + transform
X_test  = scaler.transform(X_test)       # transform만!
```

스케일링 예제

- “week10_example.csv”를 사용하세요.
- 1) 이상치 제거 후 StandardScaler 적용 시 나이_scaled 평균은?
- 2) 스케일링 후 나이_scaled 표준편차는?

10주차 과제

"week10_hospital_raw.csv" 데이터는 병원에 입원한 환자 200명의 진료 기록이다. 변수 설명은 다음과 같다.

이 데이터를 이용하여 Eclass에 10주차 과제를 푸세요.

변수	설명
환자ID	환자 고유 번호
나이	환자 나이 (세)
체중	환자 체중 (kg)
키	환자 키 (cm)
혈압	수축기 혈압 (mmHg)
혈당	공복 혈당 (mg/dL)
진료비	진료비 (원)
성별	환자 성별 (남/여)
등급	환자 등급 (BRONZE/SILVER/GOLD/VIP)
진단	진단 결과 (정상/주의/위험)
입원일	입원 날짜 (YYYY-MM-DD)